Professional Guide to

AI-Powered Real-World Projects

Project 9 of 20

Full-Stack Architecture, Product Design, AI Integration, Testing, Deployment, and Viva Preparation

Purpose of This Guide

This document helps students move from a project title to a production-minded implementation. Every chapter defines the problem, users, scope, workflows, architecture, database, APIs, AI boundaries, security, tests, milestones, deliverables, and evaluation criteria.

Student Project Development Edition

Academic Year 2026

How to Use This Book

Choose one project only after reading its problem definition, minimum viable product, risks, and semester plan. Start with deterministic business rules and a reliable full-stack workflow before adding AI features. Treat every AI output as untrusted input that must be validated, stored with provenance, and reviewed when the decision can affect a person. Use the chapter checklists during weekly reviews and include evidence such as diagrams, API tests, screenshots, logs, and deployment links in the final submission.

Recommended Team Responsibilities

Responsibility	Expected Ownership
Product Lead	Problem research, scope, user stories, acceptance criteria, demo narrative, and stakeholder communication.
Frontend Lead	Design system, responsive screens, accessibility, client state, API integration, and usability testing.
Backend Lead	Domain model, APIs, authorization, validation, jobs, integrations, logging, and deployment readiness.
Data and AI Lead	Datasets, evaluation, prompts or models, safety checks, fallbacks, monitoring, and AI documentation.
Quality Lead	Test plan, defect tracking, security checks, performance evidence, release checklist, and documentation quality.

Definition of a Professional Student Project

- The product solves a specific problem for clearly defined users.
- The core workflow works end to end with persistent data and authorization.
- The interface handles loading, empty, success, validation, and failure states.
- The backend validates all input and enforces permissions independently of the frontend.
- AI is used only where it creates measurable value and has a deterministic fallback.
- The team can explain architecture choices, trade-offs, limitations, and future work.
- Tests, logs, monitoring, deployment, and documentation are treated as product features.

Contents

Project 9

AI Meeting Memory System

Executive Summary: AMMS

Domain: Organizational knowledge and meeting productivity.
Problem: Decisions, context, action items, and ownership are lost across recordings, notes, chat messages, and employee memory.
Proposed product: A permission-aware meeting workspace that transcribes recordings, produces traceable summaries, extracts actions, and enables semantic retrieval.

9.1 Why This Project Matters

This project is suitable for a major academic submission because it combines product thinking, user experience, backend engineering, data modeling, security, analytics, and responsible AI. A strong implementation should demonstrate one complete operational journey instead of presenting disconnected screens or static dashboard values. The project must be evaluated by the quality of its decisions, evidence, and user outcomes rather than by the number of libraries included.

9.1.1 Real-World Context

Decisions, context, action items, and ownership are lost across recordings, notes, chat messages, and employee memory. Existing informal processes usually depend on spreadsheets, chat groups, manual follow-up, or personal networks. Those approaches are difficult to search, difficult to audit, and difficult to scale across a department or institution. A permission-aware meeting workspace that transcribes recordings, produces traceable summaries, extracts actions, and enables semantic retrieval.

9.1.2 Value Proposition

- For end users, the platform reduces confusion and makes organizational knowledge and meeting productivity workflows easier to complete.
- For operational teams, the platform creates consistent records, queues, decisions, and reports.
- For administrators, the platform provides measurable activity and outcome indicators without relying on manual consolidation.

- For students building the system, the project demonstrates end-to-end engineering and defensible product judgment.

9.2 Problem Definition and Research Plan

Research Area	Questions to Answer Before Development
User behavior	How do people currently handle organizational knowledge and meeting productivity, and where do they lose the most time or trust?
Current tools	Which spreadsheets, forms, chat groups, portals, or manual approvals are used today?
Decision rules	Which decisions are objective, which require human judgment, and which must never be delegated to AI?
Data availability	Which fields are already available, who owns them, and how accurate or sensitive are they?
Success	Which behavior proves that AI Meeting Memory System is useful after four, eight, and twelve weeks?
Adoption	What would prevent a user from completing the main workflow, and what trust signals are required?

9.2.1 Suggested Discovery Activities

1. Interview at least five target users from two different roles.
2. Observe one current workflow from beginning to end.
3. Create a problem map that separates symptoms from root causes.
4. Write the top ten assumptions and mark each as verified, uncertain, or false.
5. Create a low-fidelity prototype and test it before designing the final interface.
6. Define baseline measurements so improvement can be demonstrated after the pilot.

9.3 Scope and Product Boundaries

9.3.1 Minimum Viable Product

- Secure access for Team Member, Meeting Owner, Workspace Manager, Organization Administrator.
- A complete workflow across Workspace, Meeting Capture, Transcription, Summary, Action Items.
- Persistent records with ownership, timestamps, validation, and lifecycle status.
- A role-specific dashboard driven by real backend data.

- At least one useful intelligence feature: speech-to-text.
- An administrator workflow for configuration, review, audit, or support.
- A deployed demonstration environment with seeded sample data.

9.3.2 Out of Scope for the First Release

- Native mobile applications unless the team already has a stable responsive web version.
- Microservices before the modular monolith has proven performance or ownership limits.
- Fully autonomous high-impact decisions based only on AI output.
- Complex payment settlement, biometric surveillance, or legal identity verification without institutional support.
- Large third-party integrations that do not improve the primary demonstration workflow.
- Features that cannot be tested with available users, data, time, or infrastructure.

9.4 Users, Roles, and Permissions

Role	Core Responsibilities
Team Member	register and complete a verified profile; view a personalized dashboard; and receive status and deadline notifications.
Meeting Owner	sign in securely and recover account access; create and update owned records; and review an activity history.
Workspace Manager	view a personalized dashboard; search and filter permitted records; and export or share an authorized report.
Organization Administrator	create and update owned records; receive status and deadline notifications; and register and complete a verified profile.

9.4.1 Authorization Rules

- A user can read or modify a private record only when ownership or an explicit role grant permits it.
- Administrative access must be checked on the server and recorded in an audit trail.
- Public pages must expose an allowlisted view model rather than the full database object.
- File access must use signed or permission-checked URLs instead of permanent public links.
- Role changes, verification decisions, exports, and destructive actions require audit events.
- Suspended users retain historical records but cannot create new activity.

9.5 Functional Requirements

9.5.1 FR-01: Workspace

1. The system shall provide a clear entry point for the workspace workflow.
2. The backend shall validate every workspace request before changing persistent state.
3. The interface shall show loading, empty, success, validation, permission, and server-error states for workspace.
4. Authorized users shall be able to search, filter, sort, or review relevant workspace records.
5. Important workspace state changes shall create timestamps, actor references, and audit events.
6. The module shall publish domain events when another module needs notification or analytics updates.

9.5.2 FR-02: Meeting Capture

1. The system shall provide a clear entry point for the meeting capture workflow.
2. The backend shall validate every meeting capture request before changing persistent state.
3. The interface shall show loading, empty, success, validation, permission, and server-error states for meeting capture.
4. Authorized users shall be able to search, filter, sort, or review relevant meeting capture records.
5. Important meeting capture state changes shall create timestamps, actor references, and audit events.
6. The module shall publish domain events when another module needs notification or analytics updates.

9.5.3 FR-03: Transcription

1. The system shall provide a clear entry point for the transcription workflow.
2. The backend shall validate every transcription request before changing persistent state.
3. The interface shall show loading, empty, success, validation, permission, and server-error states for transcription.
4. Authorized users shall be able to search, filter, sort, or review relevant transcription records.
5. Important transcription state changes shall create timestamps, actor references, and audit events.
6. The module shall publish domain events when another module needs notification or analytics updates.

9.5.4 FR-04: Summary

1. The system shall provide a clear entry point for the summary workflow.

2. The backend shall validate every summary request before changing persistent state.
3. The interface shall show loading, empty, success, validation, permission, and server-error states for summary.
4. Authorized users shall be able to search, filter, sort, or review relevant summary records.
5. Important summary state changes shall create timestamps, actor references, and audit events.
6. The module shall publish domain events when another module needs notification or analytics updates.

9.5.5 FR-05: Action Items

1. The system shall provide a clear entry point for the action items workflow.
2. The backend shall validate every action items request before changing persistent state.
3. The interface shall show loading, empty, success, validation, permission, and server-error states for action items.
4. Authorized users shall be able to search, filter, sort, or review relevant action items records.
5. Important action items state changes shall create timestamps, actor references, and audit events.
6. The module shall publish domain events when another module needs notification or analytics updates.

9.5.6 FR-06: Semantic Search

1. The system shall provide a clear entry point for the semantic search workflow.
2. The backend shall validate every semantic search request before changing persistent state.
3. The interface shall show loading, empty, success, validation, permission, and server-error states for semantic search.
4. Authorized users shall be able to search, filter, sort, or review relevant semantic search records.
5. Important semantic search state changes shall create timestamps, actor references, and audit events.
6. The module shall publish domain events when another module needs notification or analytics updates.

9.5.7 FR-07: Sharing

1. The system shall provide a clear entry point for the sharing workflow.
2. The backend shall validate every sharing request before changing persistent state.
3. The interface shall show loading, empty, success, validation, permission, and server-error states for sharing.
4. Authorized users shall be able to search, filter, sort, or review relevant sharing records.

5. Important sharing state changes shall create timestamps, actor references, and audit events.
6. The module shall publish domain events when another module needs notification or analytics updates.

9.5.8 FR-08: Retention Controls

1. The system shall provide a clear entry point for the retention controls workflow.
2. The backend shall validate every retention controls request before changing persistent state.
3. The interface shall show loading, empty, success, validation, permission, and server-error states for retention controls.
4. Authorized users shall be able to search, filter, sort, or review relevant retention controls records.
5. Important retention controls state changes shall create timestamps, actor references, and audit events.
6. The module shall publish domain events when another module needs notification or analytics updates.

9.6 Non-Functional Requirements

Quality Attribute	Professional Requirement
Security	Protect identity, authorization boundaries, secrets, uploads, and sensitive records.
Privacy	Collect only necessary data and expose it only to explicitly permitted roles.
Performance	Keep common reads responsive and move expensive processing to background jobs.
Scalability	Support growth through stateless APIs, indexes, caching, queues, and object storage.
Reliability	Use validation, idempotency, retries, backups, health checks, and clear failure states.
Accessibility	Meet keyboard, contrast, semantic markup, focus, and screen-reader expectations.
Maintainability	Keep modules cohesive, interfaces documented, and business rules covered by tests.
Observability	Record structured logs, metrics, traces, audit events, and actionable alerts.

9.6.1 Suggested Service Targets

- Ninety-fifth percentile API response below 500 ms for indexed read operations in the pilot environment.

- User-facing write operations should acknowledge success or failure within two seconds unless intentionally asynchronous.
- Background AI or media jobs must expose queued, processing, completed, failed, and retrying states.
- The system should support at least 100 concurrent pilot users without data corruption or authorization leakage.
- Daily database backups and a documented restore exercise are required before the final demonstration.
- Critical accessibility flows must be usable with keyboard navigation and visible focus indicators.

9.7 End-to-End User Workflows

9.7.1 Primary Success Flow

1. A Team Member creates an account and completes the required profile fields.
2. The user starts the meeting capture workflow and submits validated information.
3. The system creates or updates a Meeting record and records ownership.
4. The action items logic evaluates the record using transparent rules.
5. A Meeting Owner reviews or responds when human action is required.
6. The system creates a TranscriptSegment or equivalent outcome record.
7. The user receives a notification and sees the updated status on the dashboard.
8. Analytics update the selected measures, including transcription completion, action completion, search success.

9.7.2 Exception Flows

- Incomplete input is rejected with field-level guidance and no partial domain record.
- Duplicate submissions return an idempotent response or a clear conflict message.
- Unauthorized access returns a generic permission error and creates a security event when appropriate.
- Unavailable AI services trigger a retry or deterministic fallback without blocking the complete product.
- Rejected or disputed decisions preserve history and expose the permitted appeal or correction path.
- Failed notifications do not roll back the underlying business transaction.

9.8 Information Architecture and Screen Plan

Screen	Required Experience
Public Landing Page	Problem, benefits, trust signals, process explanation, and primary call to action.
Authentication	Registration, login, verification, password recovery, and secure session feedback.
Team Member Dashboard	Priority tasks, recent activity, transcription completion, status summaries, and recommended next action.
Meeting Capture Workspace	Create, edit, validate, save draft, submit, and review owned information.
Transcription Screen	Focused workflow with contextual help, status visibility, and evidence display.
Action Items Results	Explain the result, source data, confidence, limitations, and available next steps.
Notifications Center	Unread state, category filters, deep links, and preference controls.
Organization Administrator Console	Configuration, review queues, audit history, metrics, and support tools.

9.8.1 Frontend State Checklist

- Initial loading skeleton
- No-data onboarding state
- Successful data state
- Field validation state
- Permission-denied state
- Network or server error state
- Background processing state
- Partial data warning
- Confirmation for destructive actions
- Mobile navigation and small-screen tables

9.9 Recommended Technical Architecture

9.9.1 Architecture Style

Use a modular monolith for the first production-minded version. Keep domain modules independent inside one deployable backend, and separate long-running AI, media, or notification work through a

queue. This approach is easier to test and deploy than premature microservices while still creating clear boundaries for future extraction.

9.9.2 Suggested Technology Stack

Layer	Recommended Choice
Web application	React with Vite or Next.js, TypeScript, Tailwind CSS, React Query, React Hook Form, and an accessible component system.
API	Node.js with Express or NestJS, TypeScript, Zod or Joi validation, structured errors, and OpenAPI documentation.
Database	PostgreSQL for relational integrity, or MongoDB when the team can justify document-shaped aggregates and indexes.
Cache and jobs	Redis for short-lived cache, rate limits, distributed locks, and a BullMQ-compatible background queue.
Files	S3-compatible object storage or Cloudinary with signed uploads, MIME validation, size limits, and retention rules.
AI boundary	A dedicated service module that validates structured output, records model metadata, and supports fallbacks.
Deployment	Docker containers, managed database, environment-based configuration, HTTPS, logs, metrics, and automated migrations.

9.9.3 Logical Component Flow

Listing 9.1: Logical request and processing flow

```
Browser / Mobile Web
       |
       v
API Gateway and Authentication Middleware
       |
       v
Domain Service -> Repository -> Primary Database
       |
       +--> Event / Job Queue --> Worker --> AI or External Service
       | |
       +<------ Status and Result ----+
       |
       +--> Notification Service
       +--> Analytics Event Store
       +--> Audit Log
```

9.10 Frontend Engineering Blueprint

9.10.1 Suggested Folder Structure

Listing 9.2: Feature-oriented frontend structure

```
frontend/
  src/
    app/
      router/
      providers/
      layouts/
    components/
      ui/
      feedback/
      navigation/
    features/
      amms/
        api/
        components/
        hooks/
        pages/
        schemas/
        types/
    lib/
      http/
      auth/
      validation/
      analytics/
    styles/
    tests/
```

9.10.2 Frontend Implementation Rules

- Use server state tooling for API data and keep local UI state separate.
- Validate forms on the client for usability and again on the server for security.
- Centralize API errors into user-friendly messages while preserving request identifiers for debugging.
- Use route-level authorization only as a convenience; the backend remains the authority.
- Build reusable accessible primitives for buttons, inputs, dialogs, tables, status badges, and empty states.
- Measure major flows with privacy-conscious analytics events.
- Never expose API keys, database credentials, private prompts, or privileged service tokens in frontend code.

9.11 Backend Engineering Blueprint

9.11.1 Suggested Folder Structure

Listing 9.3: Modular backend structure

```
backend/
  src/
```

```
  config/
  common/
    auth/
    errors/
    logging/
    validation/
  modules/
    workspace/
    meeting-capture/
    transcription/
    summary/
    action-items/
    semantic-search/
    sharing/
    retention-controls/
  jobs/
  integrations/
  database/
    migrations/
    seeds/
  app.ts
  server.ts
tests/
Dockerfile
```

9.11.2 Backend Rules

- Controllers translate HTTP input and output; domain services contain business rules.
- Repositories isolate database access and make query behavior testable.
- Validation schemas are shared by handlers and API documentation where practical.
- Business errors use stable machine-readable codes and safe user messages.
- Write endpoints accept idempotency keys when duplicate actions would be harmful.
- External integrations use timeouts, limited retries, circuit-breaking behavior, and request correlation identifiers.
- Secrets are loaded from environment or a secret manager and are never committed to Git.

9.12 Database Design

Entity	Purpose
User	Stores user information for workspace, including identifiers, lifecycle state, ownership, and timestamps.
Workspace	Stores workspace information for meeting capture, including identifiers, lifecycle state, ownership, and timestamps.

Entity	Purpose
Meeting	Stores meeting information for transcription, including identifiers, lifecycle state, ownership, and timestamps.
Participant	Stores participant information for summary, including identifiers, lifecycle state, ownership, and timestamps.
Recording	Stores recording information for action items, including identifiers, lifecycle state, ownership, and timestamps.
TranscriptSegment	Stores transcriptsegment information for semantic search, including identifiers, lifecycle state, ownership, and timestamps.
Summary	Stores summary information for sharing, including identifiers, lifecycle state, ownership, and timestamps.
ActionItem	Stores actionitem information for retention controls, including identifiers, lifecycle state, ownership, and timestamps.
SearchDocument	Stores searchdocument information for workspace, including identifiers, lifecycle state, ownership, and timestamps.
AccessPolicy	Stores accesspolicy information for meeting capture, including identifiers, lifecycle state, ownership, and timestamps.

9.12.1 Common Record Fields

- id: stable primary identifier
- organizationId or campusId: tenant boundary when multiple institutions are supported
- createdBy and updatedBy: actor references
- createdAt and updatedAt: server-generated timestamps
- status: controlled lifecycle state
- version: optimistic concurrency or revision number
- deletedAt: optional soft-delete timestamp for recoverable records
- metadata: limited extension object with documented keys

9.12.2 Indexing Plan

- Unique index on normalized email and other true business identifiers.
- Compound indexes for the most common tenant, status, owner, and date filters.
- Text or search-engine index only for fields intended for discovery.
- Time-based index for events, notifications, audits, and analytics rollups.
- Foreign-key or reference indexes for every frequently joined relation.

- A query plan check for dashboard and search endpoints before the final release.

9.13 API Design

Method	Endpoint	Responsibility
POST	/api/v1/amms/auth/register	Create an account and required profile seed.
POST	/api/v1/amms/auth/login	Create a secure authenticated session.
GET	/api/v1/amms/me	Return the current user's safe profile view.
PATCH	/api/v1/amms/me	Update allowlisted profile fields.
GET	/api/v1/amms/meeting	List authorized Meeting records with filters and pagination.
POST	/api/v1/amms/meeting	Create and validate a Meeting record.
GET	/api/v1/amms/meeting/:id	Read an authorized Meeting detail view.
PATCH	/api/v1/amms/meeting/:id	Update mutable Meeting fields.
POST	/api/v1/amms/transcriptsegment	Start the Action Items or review workflow.
GET	/api/v1/amms/dashboard	Return role-specific summaries including transcription completion.
GET	/api/v1/amms/notifications	List user notifications using cursor pagination.
GET	/api/v1/amms/admin/audit-events	Return permission-checked audit history.

9.13.1 API Response Standard

Listing 9.4: Example successful response envelope

```
{
  "data": { "id": "record-id", "status": "active" },
  "meta": { "requestId": "req-id", "timestamp": "ISO-8601" },
  "error": null
}
```

Listing 9.5: Example validation error envelope

```
{
  "data": null,
  "meta": { "requestId": "req-id", "timestamp": "ISO-8601" },
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Please correct the highlighted fields.",
    "details": [{ "field": "title", "reason": "required" }]
  }
}
```

9.14 Scoring, Matching, or Decision Logic

Begin with a deterministic and explainable baseline. A transparent baseline is easier to test, demonstrate, and improve than an opaque AI-only score. Store each component score and explanation so that users and reviewers can understand the result.

Component	Example Weight and Interpretation
Profile or input completeness	15 percent; enough reliable data exists to perform the workflow.
Core eligibility or relevance	30 percent; mandatory domain requirements are satisfied.
Evidence or quality strength	25 percent; claims are supported by recent and relevant evidence.
Compatibility or operational fit	20 percent; preferences, constraints, status, and availability align.
Trust and recency	10 percent; verified, recent, and non-disputed information receives greater confidence.

Scoring Rule

> Weights are examples, not universal truth. The team must derive final weights from interviews, pilot evidence, and stakeholder review. High-impact decisions require a human-readable explanation and an appeal or correction path.

9.15 AI Integration Plan

9.15.1 Useful AI Capabilities

- speech-to-text: use a constrained input contract, structured output schema, and measurable evaluation set.
- speaker-aware summarization: use a constrained input contract, structured output schema, and measurable evaluation set.
- action extraction: use a constrained input contract, structured output schema, and measurable evaluation set.
- topic segmentation: use a constrained input contract, structured output schema, and measurable evaluation set.
- retrieval-augmented question answering: use a constrained input contract, structured output schema, and measurable evaluation set.

9.15.2 Responsible AI Pipeline

1. Confirm that the user has permission to process the selected data.

2. Remove unnecessary personal or secret information before sending data to a model.
3. Build a versioned prompt or model request with explicit task, context, constraints, and output schema.
4. Apply timeouts, rate limits, and cost controls.
5. Validate the response against a strict schema.
6. Reject or repair malformed output without trusting free-form text as executable instructions.
7. Apply domain rules and safety checks after model processing.
8. Store model name, request version, timestamp, confidence indicators, and reviewer corrections.
9. Display limitations and provide a non-AI fallback or manual review path.
10. Evaluate accuracy and usefulness on a representative test set before release.

9.15.3 AI Evaluation Dataset

- At least 50 anonymized or synthetic examples representing normal inputs.
- Edge cases with missing, contradictory, multilingual, or low-quality input.
- Adversarial text attempting prompt injection or policy bypass.
- Examples from different user groups to inspect uneven quality.
- Expected structured outputs reviewed by a domain-aware human.
- A failure log that distinguishes model errors, prompt errors, data errors, and product-rule errors.

9.16 Security, Privacy, and Ethics

Risk	Required Control
Account takeover	Strong password hashing, secure cookies or carefully protected tokens, rate limiting, verification, and session revocation.
Broken access control	Central authorization policies, ownership checks, tenant checks, and negative authorization tests.
Unsafe input	Schema validation, sanitization where needed, parameterized queries, and safe file processing.
Sensitive file exposure	Private object storage, signed URLs, malware scanning where available, and retention controls.
Data over-collection	Field-level purpose review, consent, minimization, deletion, and export procedures.
AI data leakage	Redaction, provider settings review, prompt isolation, output validation, and prohibited-data rules.

Risk	Required Control
Abuse and fraud	Reporting, moderation queues, velocity limits, evidence review, suspension, and appeals.
Audit gaps	Immutable or append-oriented security events with actor, action, target, time, and request identifier.

9.16.1 Project-Specific Risk Register

9.16.1.1 Risk 1: recording requires participant consent

- Define the likelihood and impact before implementation.
- Add a preventive product or technical control.
- Add a detection signal and responsible owner.
- Document the user-facing correction, appeal, or recovery path.
- Retest the risk during pilot review.

9.16.1.2 Risk 2: hallucinated summaries can misrepresent decisions

- Define the likelihood and impact before implementation.
- Add a preventive product or technical control.
- Add a detection signal and responsible owner.
- Document the user-facing correction, appeal, or recovery path.
- Retest the risk during pilot review.

9.16.1.3 Risk 3: workspace permissions must prevent data leakage

- Define the likelihood and impact before implementation.
- Add a preventive product or technical control.
- Add a detection signal and responsible owner.
- Document the user-facing correction, appeal, or recovery path.
- Retest the risk during pilot review.

9.17 Analytics and Success Measurement

Metric	Definition and Use
transcription completion	Define transcription completion with an exact numerator, denominator, event source, reporting period, and owner. Use it to evaluate whether AI Meeting Memory System improves real behavior rather than only page views.
action completion	Define action completion with an exact numerator, denominator, event source, reporting period, and owner. Use it to evaluate whether AI Meeting Memory System improves real behavior rather than only page views.
search success	Define search success with an exact numerator, denominator, event source, reporting period, and owner. Use it to evaluate whether AI Meeting Memory System improves real behavior rather than only page views.
summary correction rate	Define summary correction rate with an exact numerator, denominator, event source, reporting period, and owner. Use it to evaluate whether AI Meeting Memory System improves real behavior rather than only page views.
weekly active teams	Define weekly active teams with an exact numerator, denominator, event source, reporting period, and owner. Use it to evaluate whether AI Meeting Memory System improves real behavior rather than only page views.

9.17.1 Analytics Events

- account_registered
- profile_completed
- meeting_capture_started
- transcription_submitted
- action_items_completed
- result_viewed
- recommendation_accepted
- notification_opened
- report_exported
- support_or_appeal_created

9.18 Testing Strategy

9.18.1 TC-01: authentication and session handling

- Create a positive test for authentication and session handling.
- Create a negative or unauthorized test for authentication and session handling.
- Verify persisted data and side effects after authentication and session handling.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.2 TC-02: role-based authorization

- Create a positive test for role-based authorization.
- Create a negative or unauthorized test for role-based authorization.
- Verify persisted data and side effects after role-based authorization.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.3 TC-03: request payload validation

- Create a positive test for request payload validation.
- Create a negative or unauthorized test for request payload validation.
- Verify persisted data and side effects after request payload validation.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.4 TC-04: ownership enforcement

- Create a positive test for ownership enforcement.
- Create a negative or unauthorized test for ownership enforcement.
- Verify persisted data and side effects after ownership enforcement.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.5 TC-05: duplicate submission handling

- Create a positive test for duplicate submission handling.
- Create a negative or unauthorized test for duplicate submission handling.
- Verify persisted data and side effects after duplicate submission handling.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.6 TC-06: search and filter correctness

- Create a positive test for search and filter correctness.
- Create a negative or unauthorized test for search and filter correctness.

- Verify persisted data and side effects after search and filter correctness.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.7 TC-07: pagination and stable sorting

- Create a positive test for pagination and stable sorting.
- Create a negative or unauthorized test for pagination and stable sorting.
- Verify persisted data and side effects after pagination and stable sorting.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.8 TC-08: file upload validation

- Create a positive test for file upload validation.
- Create a negative or unauthorized test for file upload validation.
- Verify persisted data and side effects after file upload validation.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.9 TC-09: notification delivery

- Create a positive test for notification delivery.
- Create a negative or unauthorized test for notification delivery.
- Verify persisted data and side effects after notification delivery.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.10 TC-10: background job retry behavior

- Create a positive test for background job retry behavior.
- Create a negative or unauthorized test for background job retry behavior.
- Verify persisted data and side effects after background job retry behavior.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.11 TC-11: AI timeout and fallback behavior

- Create a positive test for AI timeout and fallback behavior.
- Create a negative or unauthorized test for AI timeout and fallback behavior.
- Verify persisted data and side effects after AI timeout and fallback behavior.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.12 TC-12: AI structured-output validation

- Create a positive test for AI structured-output validation.
- Create a negative or unauthorized test for AI structured-output validation.
- Verify persisted data and side effects after AI structured-output validation.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.13 TC-13: audit event creation

- Create a positive test for audit event creation.
- Create a negative or unauthorized test for audit event creation.
- Verify persisted data and side effects after audit event creation.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.14 TC-14: privacy redaction

- Create a positive test for privacy redaction.
- Create a negative or unauthorized test for privacy redaction.
- Verify persisted data and side effects after privacy redaction.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.15 TC-15: accessibility keyboard flow

- Create a positive test for accessibility keyboard flow.
- Create a negative or unauthorized test for accessibility keyboard flow.
- Verify persisted data and side effects after accessibility keyboard flow.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.16 TC-16: responsive layout

- Create a positive test for responsive layout.
- Create a negative or unauthorized test for responsive layout.
- Verify persisted data and side effects after responsive layout.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.17 TC-17: empty and loading states

- Create a positive test for empty and loading states.
- Create a negative or unauthorized test for empty and loading states.

- Verify persisted data and side effects after empty and loading states.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.18 TC-18: error recovery

- Create a positive test for error recovery.
- Create a negative or unauthorized test for error recovery.
- Verify persisted data and side effects after error recovery.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.19 TC-19: database index usage

- Create a positive test for database index usage.
- Create a negative or unauthorized test for database index usage.
- Verify persisted data and side effects after database index usage.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.20 TC-20: backup restoration

- Create a positive test for backup restoration.
- Create a negative or unauthorized test for backup restoration.
- Verify persisted data and side effects after backup restoration.
- Record the expected API status, UI state, audit event, and notification behavior.

9.18.21 Required Test Layers

- Unit tests for scoring, permissions, lifecycle transitions, and formatting utilities.
- Integration tests for repositories, API handlers, queues, storage, and external-service adapters.
- End-to-end tests for the primary success flow and one important failure flow.
- Contract tests for AI structured output and third-party webhook payloads.
- Accessibility checks using automated tooling plus keyboard review.
- Load tests for dashboard reads, search, and one common write endpoint.
- Backup restoration and deployment rollback rehearsal.

9.19 Detailed Product Backlog

9.19.1 US-01: account access

Story: As a Team Member, I want to manage account access so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The account access entry point is visible only to authorized roles.
2. Required account access fields have labels, help text, validation, and accessible errors.
3. A successful account access action persists data and displays a confirmation.
4. A failed account access action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed account access requests.
6. Important account access changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

9.19.2 US-02: profile completion

Story: As a Meeting Owner, I want to manage profile completion so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The profile completion entry point is visible only to authorized roles.
2. Required profile completion fields have labels, help text, validation, and accessible errors.
3. A successful profile completion action persists data and displays a confirmation.
4. A failed profile completion action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed profile completion requests.
6. Important profile completion changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.

- No secrets, personal test data, or debug logs committed.

9.19.3 US-03: meeting capture

Story: As a Workspace Manager, I want to manage meeting capture so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The meeting capture entry point is visible only to authorized roles.
2. Required meeting capture fields have labels, help text, validation, and accessible errors.
3. A successful meeting capture action persists data and displays a confirmation.
4. A failed meeting capture action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed meeting capture requests.
6. Important meeting capture changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

9.19.4 US-04: transcription

Story: As a Organization Administrator, I want to manage transcription so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The transcription entry point is visible only to authorized roles.
2. Required transcription fields have labels, help text, validation, and accessible errors.
3. A successful transcription action persists data and displays a confirmation.
4. A failed transcription action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed transcription requests.
6. Important transcription changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.

- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

9.19.5 US-05: summary

Story: As a Team Member, I want to manage summary so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The summary entry point is visible only to authorized roles.
2. Required summary fields have labels, help text, validation, and accessible errors.
3. A successful summary action persists data and displays a confirmation.
4. A failed summary action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed summary requests.
6. Important summary changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

9.19.6 US-06: action items

Story: As a Meeting Owner, I want to manage action items so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The action items entry point is visible only to authorized roles.
2. Required action items fields have labels, help text, validation, and accessible errors.
3. A successful action items action persists data and displays a confirmation.
4. A failed action items action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed action items requests.
6. Important action items changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.

- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

9.19.7 US-07: semantic search

Story: As a Workspace Manager, I want to manage semantic search so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The semantic search entry point is visible only to authorized roles.
2. Required semantic search fields have labels, help text, validation, and accessible errors.
3. A successful semantic search action persists data and displays a confirmation.
4. A failed semantic search action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed semantic search requests.
6. Important semantic search changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

9.19.8 US-08: sharing

Story: As a Organization Administrator, I want to manage sharing so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The sharing entry point is visible only to authorized roles.
2. Required sharing fields have labels, help text, validation, and accessible errors.
3. A successful sharing action persists data and displays a confirmation.
4. A failed sharing action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed sharing requests.
6. Important sharing changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.

- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

9.19.9 US-09: notifications

Story: As a Team Member, I want to manage notifications so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The notifications entry point is visible only to authorized roles.
2. Required notifications fields have labels, help text, validation, and accessible errors.
3. A successful notifications action persists data and displays a confirmation.
4. A failed notifications action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed notifications requests.
6. Important notifications changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

9.19.10 US-10: search and filters

Story: As a Meeting Owner, I want to manage search and filters so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The search and filters entry point is visible only to authorized roles.
2. Required search and filters fields have labels, help text, validation, and accessible errors.
3. A successful search and filters action persists data and displays a confirmation.
4. A failed search and filters action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed search and filters requests.
6. Important search and filters changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

9.19.11 US-11: reports

Story: As a Workspace Manager, I want to manage reports so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The reports entry point is visible only to authorized roles.
2. Required reports fields have labels, help text, validation, and accessible errors.
3. A successful reports action persists data and displays a confirmation.
4. A failed reports action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed reports requests.
6. Important reports changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

9.19.12 US-12: administration

Story: As a Organization Administrator, I want to manage administration so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The administration entry point is visible only to authorized roles.
2. Required administration fields have labels, help text, validation, and accessible errors.
3. A successful administration action persists data and displays a confirmation.
4. A failed administration action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed administration requests.
6. Important administration changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

9.20 Implementation Roadmap

9.20.1 Phase 0: Discovery

Define users, assumptions, success measures, constraints, and out-of-scope behavior. **Exit evidence:**

- A reviewed artifact for phase 0: discovery is stored in the project documentation.
- At least one demonstrable AI Meeting Memory System workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

9.20.2 Phase 1: Foundation

Set up repositories, environments, design tokens, routing, API conventions, and database access. **Exit evidence:**

- A reviewed artifact for phase 1: foundation is stored in the project documentation.
- At least one demonstrable AI Meeting Memory System workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

9.20.3 Phase 2: Identity

Implement authentication, authorization, profile completion, and account security. **Exit evidence:**

- A reviewed artifact for phase 2: identity is stored in the project documentation.
- At least one demonstrable AI Meeting Memory System workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

9.20.4 Phase 3: Core Records

Build the primary create, read, update, archive, and search workflows. **Exit evidence:**

- A reviewed artifact for phase 3: core records is stored in the project documentation.
- At least one demonstrable AI Meeting Memory System workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

9.20.5 Phase 4: Collaboration

Add requests, reviews, comments, notifications, and role-specific work queues. **Exit evidence:**

- A reviewed artifact for phase 4: collaboration is stored in the project documentation.
- At least one demonstrable AI Meeting Memory System workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

9.20.6 Phase 5: Intelligence

Implement deterministic scoring first, then add AI assistance behind a stable service boundary. **Exit evidence:**

- A reviewed artifact for phase 5: intelligence is stored in the project documentation.
- At least one demonstrable AI Meeting Memory System workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

AI Gate

> Do not move the AI feature into the primary user flow until its evaluation results, failure behavior, privacy handling, and fallback are documented.

9.20.7 Phase 6: Analytics

Create operational dashboards, event tracking, reports, and export workflows. **Exit evidence:**

- A reviewed artifact for phase 6: analytics is stored in the project documentation.
- At least one demonstrable AI Meeting Memory System workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

9.20.8 Phase 7: Hardening

Complete threat review, accessibility checks, performance testing, and recovery procedures. **Exit evidence:**

- A reviewed artifact for phase 7: hardening is stored in the project documentation.
- At least one demonstrable AI Meeting Memory System workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

9.20.9 Phase 8: Pilot

Release to a controlled cohort, measure behavior, collect feedback, and correct product assumptions. **Exit evidence:**

- A reviewed artifact for phase 8: pilot is stored in the project documentation.
- At least one demonstrable AI Meeting Memory System workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

9.20.10 Phase 9: Production

Deploy with monitoring, support ownership, release notes, and a rollback strategy. **Exit evidence:**

- A reviewed artifact for phase 9: production is stored in the project documentation.
- At least one demonstrable AI Meeting Memory System workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

9.21 Semester Execution Plan

Week	Primary Outcome
Week 1	Problem research, stakeholder interviews, and baseline measurement
Week 2	Personas, journey map, scope, risks, and success metrics
Week 3	Wireframes, design system, domain model, and API contract
Week 4	Project setup, authentication, authorization, and CI checks
Week 5	Implement Meeting Capture and Transcription
Week 6	Implement Summary and Action Items
Week 7	Implement Semantic Search and notifications
Week 8	Complete role dashboards, search, filters, and audit history

Week	Primary Outcome
Week 9	Build and evaluate speech-to-text
Week 10	Integrate AI fallback, analytics, reports, and exports
Week 11	Security review, accessibility checks, performance testing, and bug fixing
Week 12	Pilot deployment, user testing, documentation, presentation, and final demo rehearsal

9.22 Deployment and Operations

1. Create separate development, test, staging, and production configuration.
2. Build reproducible frontend and backend artifacts.
3. Run database migrations through a versioned deployment step.
4. Use managed HTTPS, database backups, private storage, and environment secrets.
5. Add health, readiness, and dependency checks.
6. Collect structured application logs with request identifiers.
7. Monitor API errors, latency, job failures, AI cost, and storage usage.
8. Seed demonstration data without using real sensitive student records.
9. Document rollback steps for application and database changes.
10. Assign an owner for production incidents and user support.

9.22.1 Environment Variables

Listing 9.6: Illustrative environment configuration

```
NODE_ENV=production
PORT=5000
DATABASE_URL=managed-database-connection
REDIS_URL=managed-redis-connection
SESSION_SECRET=secret-manager-reference
OBJECT_STORAGE_BUCKET=private-bucket-name
AI_PROVIDER_API_KEY=secret-manager-reference
AI_MODEL=approved-model-name
APP_ORIGIN=https://student-project.example
LOG_LEVEL=info
```

9.23 Documentation Deliverables

- Problem statement and evidence from user research
- Personas and user journey

- Scope, assumptions, and out-of-scope list
- Functional and non-functional requirements
- System context and component diagrams
- Entity relationship or document model diagram
- API specification with example requests and errors
- Security, privacy, and risk register
- AI model card or AI feature note
- Test plan and execution evidence
- Deployment guide and environment variable reference
- User manual and administrator manual
- Known limitations and future roadmap

9.24 Demonstration Script

1. Introduce the real problem in one minute: Decisions, context, action items, and ownership are lost across recordings, notes, chat messages, and employee memory.
2. Show the Team Member onboarding and profile experience.
3. Create or submit a real Meeting record.
4. Demonstrate Action Items with a visible explanation of its logic.
5. Switch to the Meeting Owner role and complete the required response or review.
6. Return to the original user and show notification, status, and history.
7. Show an analytics change for transcription completion.
8. Demonstrate one validation error and one permission boundary.
9. Explain the AI limitation and fallback.
10. Close with measured outcomes, known limitations, and the next product milestone.

9.25 Viva and Interview Preparation

1. What evidence shows that AI Meeting Memory System solves a real problem?
2. Why did you choose a modular monolith instead of microservices?
3. How is transcription completion calculated and validated?
4. Which fields belong to Meeting, and why?

5. How does the backend enforce role and ownership permissions?
6. What happens when the AI provider is unavailable or returns invalid data?
7. How did you evaluate AI accuracy, bias, usefulness, and cost?
8. Which database indexes support the most important queries?
9. How do you prevent duplicate writes and inconsistent state?
10. Which data is sensitive, and how can a user correct or delete it?
11. What tests protect the primary workflow?
12. What is the largest known limitation of the current release?

9.26 Evaluation Rubric

Area	Marks and Evidence
Problem understanding	10 marks: research quality, stakeholder clarity, and validated need.
Product design	10 marks: scope, workflows, usability, accessibility, and error-state design.
Frontend engineering	15 marks: responsive implementation, state handling, maintainability, and API integration.
Backend engineering	20 marks: domain rules, authorization, validation, data integrity, and error handling.
Data and AI	15 marks: justified use, evaluation, structured outputs, safety, fallback, and limitations.
Testing and security	15 marks: meaningful automated tests, threat controls, privacy, and evidence.
Deployment and operations	5 marks: reproducible release, monitoring, backup, and rollback awareness.
Documentation and viva	10 marks: diagrams, decisions, demonstration quality, and technical explanation.

9.27 Project Completion Checklist

- [ ] Problem validated with target users
- [ ] MVP and out-of-scope list approved
- [ ] Roles and authorization matrix implemented
- [ ] Primary workflow works end to end
- [ ] Database constraints and indexes reviewed

- [ ] API documentation matches implementation
- [ ] Loading, empty, success, and error states completed
- [ ] AI feature evaluated and fallback tested
- [ ] Security and privacy review completed
- [ ] Accessibility review completed
- [ ] Automated test suite passing
- [ ] Staging deployment verified
- [ ] Backup and rollback steps documented
- [ ] Demo data prepared
- [ ] Final report and viva answers reviewed

9.28 Conclusion

AI Meeting Memory System can become a strong professional project when the team treats it as a real product rather than a collection of pages. The strongest submission will demonstrate trustworthy data, clear role workflows, explainable action items, tested permissions, responsible AI, and measurable outcomes. Students should keep the first release focused, collect pilot evidence, and use that evidence to justify every major extension.

Cross-Project Architecture Checklist

- Can every important record be traced to an owner, actor, status, and timestamp?
- Can the backend reject every action the frontend hides from an unauthorized user?
- Can the main workflow recover from network, storage, queue, and AI failures?
- Can a reviewer understand why a score, match, recommendation, or alert was produced?
- Can the team restore data, roll back a release, and identify the cause of a production error?
- Can users correct inaccurate information and report harmful or abusive activity?
- Can the product be demonstrated without exposing real personal or secret data?

Final Advice to Student Teams

Build the smallest version that proves the full value chain. A complete, secure, understandable workflow is more impressive than twenty unfinished features. Keep weekly evidence: interview notes, diagrams, commits, tests, screenshots, evaluation results, and deployment logs. During the final presentation, explain not only what works but also why it was designed that way, where it can fail, and how the next version would improve.